CROSS / PROBLEM 02

Problem 2: Enable and optimize BERT on TPUs with CROSS

CADathlon · `agent_pset` · AI agents explicitly allowed · 100 points · proposed duration: 8 hours

1. Introduction

Use or improve CROSS to enable encrypted BERT inference on TPUs, then optimize the implementation to achieve better benchmark results and a higher assignment score. First make the full model run correctly; then reduce latency and increase throughput while preserving accuracy and security.

The two repositories have distinct roles:

1. **Library to use and improve: CROSS.** Build your encrypted BERT implementation with CROSS and execute its server computation on TPUs. Extend or optimize CROSS's operators, kernels, packing and execution support as needed. Submit library changes in `cross.patch`.
2. **Benchmark to evaluate your results: fhe-benchmarking/BERT.** Run your CROSS implementation through this benchmark's pinned harness. Use its BERT/MRPC workload and reported accuracy, timings and communication measurements to evaluate correctness and performance. Keep the evaluation harness unchanged.

The workload is BERT-Base fine-tuned on Microsoft Research Paraphrase Corpus (MRPC) for sentence-pair classification. Your development cycle is **implement with CROSS → run on TPUs → evaluate with the BERT benchmark → optimize and measure again**. Section 4 explains how passing benchmark results contribute to the assignment's 100-point score.

Students are explicitly allowed to use AI agents for this workload. You may use coding agents and AI assistants to design, implement, debug, test, profile, optimize and document your solution, including changes to CROSS. Record the agents, models and tools used in `REPORT.md`, describe the tasks they performed, and verify generated code and results. Teams remain responsible for correctness, privacy, reproducibility and honest reporting.

The benchmark repository also contains a DESILO reference submission, which can inform your circuit design. This assignment requires a CROSS implementation on the assigned TPU hardware. Organizers measure the CROSS baseline used for performance scoring under the same evaluation conditions.

2. Problem statement

2.1 Benchmark and model used for evaluation

Use upstream commit `170bfe567545d74d0fad785a052518351d37bc93`. `src/benchmark.py` records its harness SHA-256 manifest and model revision.

Item	Required value
Checkpoint and tokenizer	google-bert/bert-base-cased-finetuned-mrpc
Checkpoint revision	`f150c1d609d1e50dd5e2e5408661cfac8339277c`
Model loader	`BertForSequenceClassification`, evaluation mode
Encoder	12 layers; hidden width 768; 12 heads of width 64; MLP width 3072
Embeddings	Cased vocabulary 28996; learned positions; two token types; LayerNorm
Input	Two MRPC sentences; paired tokenization; truncate and pad to 128 tokens
Encoder semantics	Bidirectional masked attention; biased projections; post-residual LayerNorm; GELU
Output	First-token CLS pooler with tanh, then two logits; label 0 if logit 0 > logit 1, else label 1
Evaluation data	The 408-example GLUE MRPC validation split selected by the upstream harness

The checkpoint config's architecture hint does not replace the upstream loader: use `BertForSequenceClassification`, including its trained classifier. Keep all learned weights and tokenizer settings. Disable dropout. This is an encoder classification task: it has no causal mask, KV cache or generated token loop. LayerNorm subtracts the mean and uses the checkpoint epsilon `1e-12`; it is not RMSNorm. Softmax is over unmasked keys in each head.

2.2 Privacy boundary and permitted changes

The client owns sentence text, token IDs and the secret key. As in the upstream reference, it may compute the complete BERT embedding module, including embedding LayerNorm, before encryption. The server receives encrypted embeddings, public model weights, public/evaluation keys and attention masks. **The attention mask and resulting sequence length are public in this track**; sentence content and intermediate activations are private. Keep dataset target IDs client-side; ciphertexts may use sample positions for bookkeeping.

All 12 encoder layers, the CLS pooler and classification head execute under CKKS using CROSS. Only the client decrypts final logits. No intermediate client refresh, server decryption, plaintext fallback, input/label lookup or precomputed answers. The ranked evaluator must not be able to read client files, labels or the secret key. The supplied path dictionaries document the boundary; they do not enforce filesystem isolation.

You may change packing, tensor tiling, rotations, matrix kernels, scheduling, device placement, serialization and memory reuse. You may implement declared approximations to softmax, GELU, reciprocal square root and tanh, including bootstrapping. State coefficients, intervals, iterations, level/scale budget and calibration data. Preserve the frozen model's topology and weights; no pruning, retraining, shortened sequence length or replacement network. Calibration uses organizer-approved public training examples, never held-out evaluation labels. Measure approximation error separately from HE error.

Use validated CROSS parameters providing at least 128-bit classical security. Report degree, slots, actual Q/P towers, `log2(Q)`, `log2(QP)`, security ceiling, margin, noise/secret distributions and bootstrap parameters. Fresh randomness is required for every encryption and key generation. Another backend's ring or security claim cannot establish security for the CROSS parameters used.

CROSS's current frontend does **not** automatically compile a complete BERT model. `nn.vectorize` / `packing.pack` support a narrower polynomial graph, and the ordinary activation path's square is not a valid undeclared GELU replacement. Implement missing tensor operations and an explicit depth and bootstrap schedule. Mapping-owned bootstrap execution currently supports one device per Mapping; multi-chip execution needs an explicit compatible design. Document CROSS changes in `cross.patch`.

2.3 Required implementation and milestones

Implement `src/bert_he.py` **using CROSS**, and improve the CROSS library where needed to support efficient BERT inference on TPUs. The seven executable adapters connect your implementation to the evaluation benchmark. They use the upstream stage names and positional `size` argument; five delegate to your TODOs:

Stage	Required work
`client_key_generation`	Generate CROSS keys; persist secrets in `client_private` and only public/evaluation material in `public_keys`
`server_preprocess_model`	Load every frozen parameter; encode and cache public model constants in `io/server_data`
`client_preprocess_input`	Supplied: paired tokenization into IDs, token types and attention masks
`client_encode_encrypt_input`	Client embedding module, CROSS encoding/encryption, files in `ciphertexts_upload`
`server_encrypted_compute`	All encoder layers, pooler and classifier; output ciphertexts in `ciphertexts_download`
`client_decrypt_decode`	Write one `{target_idx, he_logits}` record per input to `intermediate/decrypted_results.jsonl`
`client_postprocess`	Supplied: validate logits, restore input order, emit one binary label per line

`stages.py` defines exact paths. Internal ciphertext/key formats are yours to design and must survive separate processes; arrays of plaintext activations are not ciphertext serialization. Cache public model data only with a key covering checkpoint, ring, layout and approximation settings. Never reuse input-specific encrypted intermediates between requests.

Also implement `create_session(config, weights, *, batch, sequence_length, devices)` for local development. Its object provides `encrypt(embeddings)`, `evaluate(ciphertexts, attention_mask=mask)`, `decrypt(ciphertexts)` and `close()`. Ciphertext boundaries use canonical CROSS `Polynomial` values, or nonempty nested lists/tuples of them. Decryption returns `(batch, 2)` logits. This local interface may share the benchmark circuit; only the upstream file-based harness establishes full benchmark completion.

Work in four milestones:

1. **P4.1 — establish evaluation:** reproduce the benchmark's plaintext BERT/MRPC results and connect the CROSS submission to its stage interfaces.
2. **P4.2 — enable CROSS operations on TPUs:** implement and validate encrypted transformer operators and a full-width encoder block, extending CROSS as needed.
3. **P4.3 — enable full BERT on TPUs:** execute all 12 layers, pooler and classifier with CROSS and pass the benchmark evaluation gates below.
4. **P4.4 — improve benchmark results:** profile the working implementation, optimize CROSS and your BERT execution, and rerun the same benchmark instances. Report the latency and throughput before and after each change and the resulting performance points under section 4.

Operator/block evidence earns partial credit; it does not establish a full BERT result. A completed encrypted BERT baseline is not bundled.

2.4 Inputs, public fixtures and evaluation gates

There are six `testN.in`/`testN.out` JSON pairs. These use seeded synthetic weights, small widths and 1–3 layers. Cases cover unpadded and padded sequences, paired token types, distinct batch lanes and multilayer composition. The plaintext oracle implements learned embeddings, bidirectional attention, post-LayerNorm, exact GELU, pooler and classifier. Public checks require finite logits of the correct shape, `abs(HE−clear) <= 0.02 + 0.01*abs(clear)` and matching binary predictions. They do not measure pretrained MRPC accuracy.

Actual benchmark instances use the full checkpoint and sequence length 128:

Size argument	Name	Sentence pairs	Role
0	single	1	End-to-end diagnostic and single-label correctness
1	small	100	Ranked accuracy and performance
2	medium	200	Ranked accuracy and performance

Size argument	Name	Sentence pairs	Role
3	large	408	Ranked accuracy and performance on the complete validation split

For size 0, upstream compares the prediction with the ground-truth label. For sizes 1–3, require **at least 75% label accuracy**, matching upstream's threshold. This contest additionally allows an accuracy drop of at most **2 percentage points** against the original cleartext checkpoint on exactly the same ordered inputs. Record prediction agreement as a diagnostic. No missing, duplicated or extra samples; all logits must be finite, all output labels binary, and predictions must agree with decrypted logits.

Run `check_result.py` after every benchmark run. It enforces the additional count, identity, output and accuracy checks without modifying the harness. At the pinned revision, the harness can finish after printing a failed quality check; size-0 verification also uses `check=False`. A zero harness exit status is therefore insufficient. Its batch quality routine also does not enforce equal input/output counts. The strict checker closes these output-checking gaps; independent source and security audits remain required.

3. How to compile and run

Use the small local fixtures during implementation. Evaluate your completed TPU implementation with the pinned **FHE BERT benchmark** using the staging and harness commands below. Repeat those benchmark runs after optimization to measure progress toward a higher score.

The organizer provides a CROSS-compatible Python environment with NumPy, PyTorch, JAX and the TPU runtime, plus Transformers, Datasets and `absl-py` for the harness. Public numerical checks need only NumPy. The oracle was checked with Transformers 4.51.3; organizers freeze the complete judge environment. DESILO is not a dependency of this CROSS submission. Do not install upstream's reference-backend requirements over the supplied CROSS environment.

```
cd cadathlon/agent_pset/p2_bert/src
make all
make reference-test        # six plaintext fixtures; no model download
make test                  # your CROSS implementation; TODOs initially fail
make submit                # local ../submission.tar.gz, no upload
```

Clone and stage the pinned benchmark in a separate workspace:

```
git clone https://github.com/fhe-benchmarking/BERT.git /tmp/bert-upstream
git -C /tmp/bert-upstream checkout 170bfe567545d74d0fad785a052518351d37bc93
make benchmark-check BERT_ROOT=/tmp/bert-upstream
make benchmark-stage BERT_ROOT=/tmp/bert-upstream BERT_RUN=/tmp/bert-cross-run
export CROSS_ROOT=/path/to/your/CROSS
cd /tmp/bert-cross-run
python3 harness/run_submission.py 0 --num_runs 1 --seed 42
python3 harness/cleartext_impl.py \
  datasets/single/intermediate/test_input.txt \
  io/single/harness_model_predictions.txt mrpc
python3 submissions/check_result.py --benchmark-root . --size 0
python3 harness/run_submission.py 1 --num_runs 1 --seed 42
python3 submissions/check_result.py --benchmark-root . --size 1
```

Repeat the last two commands with sizes 2 and 3. Small/medium/large runs automatically invoke the cleartext reference. The single case needs the explicit cleartext command above for the strict checker's diagnostic. The staging helper checks all harness source hashes, copies the unchanged harness and your current source, preserves its license, and writes a manifest. It rejects an existing destination. Restage into a new directory after edits. No upstream DESILO submission is copied.

Before a ranked run, organizers provision the pinned checkpoint/tokenizer and MRPC snapshots in an isolated offline cache. Upstream `from_pretrained` does not pin a revision, so its cached `main` must resolve to the approved snapshot; hash and verify this for both the client and cleartext evaluator. Freeze the dataset revision and input selection manifest too. The initial online asset download is preparation, never a hidden timed model change.

The harness clears `io/{instance}` at startup, retains `io/server_data`, and writes `measurements/{instance}/results-1.json`. Archive inputs, predictions, logs, measurements and the strict-check result **immediately after each run**; later invocations overwrite those paths. At this revision a fixed `--seed` with `--num_runs > 1` reuses the same selection, so ranked measurements use separate one-run invocations with published seeds, e.g. 42, 43 and 44. The 408-example instance always covers the same full split. Encrypt afresh.

Use up to eight assigned TPU v6e chips on one host. Proposed limits are 512 GiB aggregate process-tree RSS, 1 TiB work storage and no device OOM; setup (key generation and model preprocessing) at most 2 hours per configuration; online steps 5–9 at most 24 hours per instance. Organizers enforce these externally and calibrate them before the event. Completing large runs during the proposed eight-hour development window is not assumed; official scoring runs may occur afterward. CPU checks earn no TPU speed credit.

Measure three successful, fresh one-run invocations per ranked size. Use the median **sum of the harness's Input preprocessing, Input encryption, Encrypted computation, Result decryption and Result postprocessing times** as online latency `T_i`; throughput is `B_i / T_i`. These outer process times include dispatch, file I/O, compilation in online stages and synchronization. Do not substitute server-reported kernel timings for them. Report setup, data generation, reference execution, bytes transferred, peak device memory and aggregate host RSS separately. The harness `Total` includes work outside the online boundary, so it is not `T_i`. Publish the cache policy and use it for both candidate and organizer baseline; optional cache warming is not free.

4. How to score

Earn correctness points by enabling BERT inference on TPUs with CROSS; earn performance points by improving its measured benchmark results. The FHE BERT benchmark supplies accuracy, timing and communication measurements. The proposed **100-point CADathlon rubric below** converts validated results and implementation evidence into the assignment score.

For each fixed workload, lower online latency means higher throughput and more performance points, up to the stated cap. Accuracy, privacy and security gates must still pass. Organizers award points from independent benchmark reruns and audits, using the same hardware and evaluation conditions.

Component	Points	Evidence
Reproducible benchmark integration	10	Pinned assets/environment, unchanged harness, stage artifacts and strict checks
CROSS operator/block progress on TPUs	20	5 each: masked attention/softmax, GELU MLP, residual/LayerNorm, complete full-width encoder block
Full BERT benchmark correctness on TPUs	45	15 each for small, medium and large passing all gates with all 12 layers, pooler and classifier encrypted
Improved benchmark performance	15	Up to 5 per passing ranked instance for reduced measured latency, using the rule below
Evidence and report	10	2 each: privacy audit, secure parameters, approximation analysis, timing/memory records, reproducibility and AI-agent disclosure

Operator credit requires secure CROSS ciphertext execution and numerical checks against the original operator on organizer-generated full-width inputs; plaintext demonstrations and the six tiny fixtures alone do not earn it. Each operator/block output must satisfy `abs(HE-clear) <= 0.02 + 0.01*abs(clear)` with finite values and correct shape. Organizers publish calibration ranges and freeze hidden checks before release. For a fully passing ranked case let `R_i = T_base_i / T_i`; its speed points are `5 * clamp(log2(R_i) / 2, 0, 1)`. A 2× speedup earns 2.5 points; 4× earns 5. A failed case or regression earns no speed points.

`T_base_i` must be an organizer-measured, correct **CROSS BERT** baseline on the same hardware, data, security target, timing boundary and cache policy. There is no measured baseline in this starter. Organizers must implement or obtain one, validate feasibility and freeze all three values before enabling the performance leaderboard. Upstream DESILO timings cannot fill this role. An uncalibrated release is a development challenge, not a scored speed contest.

Ties break by full-model correctness points, then geometric-mean throughput across all three passing ranked instances, then lower peak device memory. Submit `src/`, `cross.patch`, `REPORT.md` and `results*.json` containing raw records, hashes and reproduction commands. Include failed configurations and AI-agent usage. Exclude secret keys, datasets, pretrained weights, caches and reference answers. `make submit` creates an archive locally. The organizer's `.golden` file is a **plaintext numerical oracle**, not an encrypted solution.

5. References

1. FHE BERT evaluation benchmark, pinned harness and model definition. `params.py`, `utils.py`, `run_submission.py` and `mrpc/test.py` define sizes, threshold, stage timing and cleartext decisions used here.
2. BERT checkpoint and tokenizer and GLUE dataset.
3. CROSS library, local `jaxite_word/API_REFERENCE.md`, `he_params.py`, `packing.py`, `mapping.py` and CKKS context APIs; use the current code for supported tensor and bootstrap operations.
4. CROSS tutorial and Tong et al., Leveraging ASIC AI Chips for Homomorphic Encryption, HPCA 2026, for accelerator design and profiling background.